
Modélisation et simulation distribuée du mouvement d'un banc de poissons

Projet de Programmation Numérique Master CHPS – Sujet 7

Etudiants :

Yohann FRONT-REIGNIER
Iram MADANI-FOUATIH

Encadrants :

S. BEN-AMOR
H. BOLLORE

TABLE DES MATIÈRES

I	Introduction	1
II	Fondements Théoriques	1
III	Architecture et Implémentation	2
III-A	Version séquentielle	2
III-B	Version Parallèle	2
IV	Analyse Comparative	2
IV-A	Recherche naïve vs SpatialGrid	2
IV-B	MPI et montée en charge	3
V	Défis Rencontrés et Résolutions	3
V-A	Lancement MPI sur le cluster	3
V-B	Compatibilité binaire / environnement	3
V-C	Coût artificiellement doublé	3
V-D	Contraintes d'allocation	3
V-E	Benchmarks trop courts	3
VI	Benchmarking Complet	3
VI-A	Visualisation des runs exploitables	3
VII	Planification des campagnes sur cluster	3
VII-A	Méthode d'estimation du coût	4
VII-B	Planning prévisionnel validable	4
VII-C	Suivi de consommation	4
VIII	Discussion	4
IX	Conclusion	4
X	Références bibliographiques	5
	Références	5

Résumé—Ce rapport présente la conception et l’implémentation d’une simulation distribuée du mouvement d’un banc de poissons, basée sur le modèle des Boids de Reynolds (1987). Trois stratégies sont comparées : une version séquentielle servant de référence, une parallélisation distribuée via MPI avec échange de halos et migration inter-rangs, et un parallélisme à mémoire partagée reposant sur Kokkos. Une grille spatiale est introduite dès la version séquentielle afin de ramener la complexité de voisinage de $O(N^2)$ à un comportement proche de $O(N)$ en moyenne, produisant un gain de $4,68\times$ à $N = 800$. La version MPI atteint un speedup de $3,89\times$ à quatre processus pour $N = 5000$ boids, et le sweep global montre que ce gain progresse de $2,32\times$ pour 500 boids jusqu’à $8,99\times$ pour 5000 boids, confirmant que l’approche distribuée devient d’autant plus avantageuse que la charge de calcul est élevée. L’ensemble des expériences a été conduit sur cluster HPC, et les défis rencontrés compatibilité binaire, surcoût d’initialisation, contraintes d’allocation sont analysés et discutés.

I. INTRODUCTION

Le mouvement d’un banc de poissons ou d’un vol groupé d’oiseaux sont des phénomènes très importants à étudier. En effet, cette étude peut mener à la création de nouveaux algorithmes bio-inspirés pour les modéliser et les simuler afin de mieux comprendre certains de leurs fonctionnements et ainsi pouvoir les répliquer dans l’industrie, notamment l’aéronautique.

Le contrôle à distance de plusieurs drones ou d’essaims de drones sans contrôle humain est aujourd’hui assez répandu. De plus, il existe aussi des zones sans tour de contrôle, où les avions doivent être capables de s’auto-gérer. C’est pour répondre à ce genre de problèmes que nous nous intéressons à la simulation de systèmes existant dans le monde animal afin de pouvoir les répliquer et les utiliser.

L’étude de ces systèmes d’auto-organisation et de flocking a mené à la création du modèle des Boids par C. Reynolds en 1987, qui permet de simuler informatiquement ces comportements. Les boids suivent trois règles simples que nous détaillerons plus tard dans ce rapport.

En respectant ces trois règles, le boid calcule sa direction et sa vitesse. Chaque boid réalise ce calcul, ce qui crée des comportements dits émergents, c’est-à-dire que certains boids vont agir de sorte à créer des motifs sans avoir de contrôle central leur demandant de les produire. Il est donc difficile de prévoir le comportement d’un boid en le regardant individuellement. C’est ce qu’on appelle un système complexe.

Une grande partie des travaux actuels consiste donc à essayer d’améliorer cette simulation afin de pouvoir gérer un plus grand nombre de boids sans trop augmenter le temps de calcul. En effet, comme chaque boid doit prendre en compte ses voisins pour déterminer son comportement, le calcul peut vite devenir très coûteux lorsque la taille du banc augmente.

C’est pourquoi plusieurs approches ont été proposées pour accélérer ce genre de simulation. Certaines utilisent le calcul

parallèle sur GPU, tandis que d’autres reposent sur une répartition du calcul sur plusieurs cœurs ou sur plusieurs machines. L’idée est alors de diviser l’espace de simulation en plusieurs parties afin que chaque unité de calcul puisse traiter une portion du banc de poissons.

Cependant, cette répartition du calcul pose plusieurs difficultés. Il faut notamment gérer les interactions entre boids situés dans des zones différentes, ainsi que les échanges d’informations lorsque les individus se déplacent d’un sous-domaine à un autre. Il faut aussi conserver un comportement réaliste, sans trop dégrader la qualité de la simulation.

Dans ce travail, nous allons donc nous intéresser à la modélisation d’un banc de poissons à l’aide du modèle des boids, puis à sa simulation distribuée. L’objectif est de comprendre comment ce modèle peut être adapté à une architecture parallèle afin d’améliorer les performances de calcul tout en gardant un comportement collectif cohérent.

Pour cela, le rapport sera organisé de la manière suivante : fondements théoriques, implémentations, comparative, défis, benchmarking, puis conclusion.

II. FONDEMENTS THÉORIQUES

Nous formalisons le modèle mathématique des boids, sa complexité algorithmique et les défis liés au parallélisme.

Afin de représenter au mieux les différents comportements du banc de poissons, nous avons utilisé des formules mathématiques associées aux règles de flocking. Pour chaque boid, le calcul repose sur les interactions avec ses voisins proches, ce qui permet de déterminer sa nouvelle direction et sa nouvelle vitesse.

1) Règle 1 : Séparation

Le calcul de la séparation est nécessaire pour trouver notre vecteur de répulsion afin d’éviter les collisions. Nous avons utilisé la formule suivante :

$$\vec{s} = \frac{1}{N} \sum_{i=1}^N \frac{\vec{p} - \vec{p}_i}{|\vec{p} - \vec{p}_i|} \quad (1)$$

où :

- $\vec{p}$: position du boid courant,
- $\vec{p}_i$: position du voisin i ,
- N : nombre de voisins dans le rayon de perception,
- la force de steering finale : $\vec{F}_s = (\hat{s} \cdot v_{max} - \vec{v})$, limitée à F_{max} la force maximale.

2) Règle 2 : Alignement

On calcule la vitesse moyenne des voisins afin de suivre leur direction et pouvoir ajuster la vitesse. Nous avons utilisé la formule suivante :

$$\vec{a} = \frac{1}{N} \sum_{i=1}^N \vec{v}_i \quad (2)$$

où :

- $\vec{v}_i$: vélocité du voisin i ,
- N : nombre de voisins,
- force de steering : $\vec{F}a = (\hat{a} \cdot v_{max} - \vec{v})$, limitée à F_{max} la force maximale.

3) Règle 3 : Cohésion

On calcule le centre de masse des voisins afin de faire en sorte de diriger l'agent vers ce centre de masse.

$$\vec{c} = \frac{1}{N} \sum_{i=1}^N \vec{p}_i \quad (3)$$

où :

- $\vec{c}$: centre de masse des voisins,
- $\vec{p}_i$: position du boid voisin i ,
- N : nombre de voisins dans le rayon de perception,
- la force de steering finale : $\vec{F}c = (\vec{c} - \vec{p} \cdot v_{max} - \vec{v})$, limitée à F_{max} la force maximale.

Paramètres clés :

- perceptionRadius : rayon de détection des voisins,
- maxSpeed : vitesse maximale du boid,
- maxForce : force maximale applicable,
- separationWeight : poids de la règle de séparation,
- alignmentWeight : poids de la règle d'alignement,
- cohesionWeight : poids de la règle de cohésion.

Pour la simulation temporelle, nous utilisons une intégration d'Euler explicite afin de mettre à jour la position et la vitesse de chaque boid à chaque pas de temps :

$$\mathbf{x}_i(t + \Delta t) = \mathbf{x}_i(t) + \mathbf{v}_i(t) \cdot \Delta t \quad (4)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t) + \mathbf{a}_i(t) \cdot \Delta t \quad (5)$$

Du point de vue algorithmique, la version naïve de cette simulation a une complexité en $O(N^2)$, car chaque boid doit comparer sa position avec tous les autres boids. Afin de réduire ce coût, on peut utiliser une grille spatiale ou une autre structure de partition de l'espace, ce qui permet en moyenne d'obtenir une complexité plus proche de $O(N)$ selon la densité de répartition des agents.

Enfin, la parallélisation de ce type de simulation pose plusieurs défis. Il faut trouver un bon compromis entre communication et calcul, gérer la synchronisation entre les différentes unités de calcul, et choisir une topologie de décomposition adaptée. En effet, si la partition de l'espace est mal choisie, les échanges entre sous-domaines peuvent devenir trop fréquents et ralentir fortement la simulation.

III. ARCHITECTURE ET IMPLÉMENTATION

A. Version séquentielle

La version séquentielle constitue notre baseline de référence : toute la simulation est exécutée dans un seul processus, sans communication réseau. Cette version permet de valider le modèle de Reynolds et de mesurer un temps de référence avant parallélisation distribuée.

L'architecture repose sur trois classes principales : `Vector2D` pour les opérations géométriques, `Boid` pour le

comportement individuel, et `Flock` pour la gestion collective. La classe `Flock` maintient la liste des boids ainsi qu'une grille spatiale (`SpatialGrid`) utilisée pour accélérer la recherche de voisins.

À chaque pas de temps, l'algorithme suit trois étapes : (1) reconstruction de la grille spatiale, (2) calcul des forces de séparation, alignement et cohésion, puis (3) intégration des positions/vitesses avec un schéma d'Euler explicite. Le wrapping toroïdal est ensuite appliqué aux frontières pour conserver les boids dans le domaine de simulation.

Cette organisation permet de réduire fortement le coût du voisinage : au lieu d'une recherche naïve en $O(N^2)$, la grille limite les comparaisons aux cellules proches, ce qui donne en pratique un comportement proche de $O(N \cdot k)$, où k est le nombre moyen de voisins pertinents.

B. Version Parallèle

La version parallèle combine un parallélisme distribué (MPI) entre processus et un parallélisme local (Kokkos/OpenMP) à l'intérieur de chaque processus. L'objectif est de conserver la cohérence physique du modèle tout en améliorant les performances quand le nombre de boids augmente.

L'espace 2D est découpé en sous-domaines, chacun géré par un rang MPI. Chaque rang ne met à jour que ses boids locaux, mais doit tenir compte des boids proches des frontières. Pour cela, un échange de halos est effectué entre rangs voisins via `MPI_Sendrecv`. Les boids qui traversent une frontière de sous-domaine sont ensuite transférés au bon rang par migration (`MPI_Alltoallv`).

Sur chaque rang, le calcul des forces et l'intégration des positions sont parallélisés avec `Kokkos::parallel_for`. Ce choix permet d'exploiter la mémoire partagée (backend OpenMP) sans modifier la logique métier des boids.

Une itération complète suit la séquence suivante : échange de halos, reconstruction de la grille locale (boids locaux + halo), calcul des forces, intégration des positions, migration inter-domaines, puis synchronisation du nombre global de boids (`MPI_Allreduce`). Cette chaîne garantit que chaque rang dispose des informations nécessaires pour calculer des interactions locales cohérentes.

IV. ANALYSE COMPARATIVE

Cette partie compare les performances des approches les plus importantes du projet à partir des mesures réellement enregistrées dans les fichiers CSV du dossier `experiments/`.

A. Recherche naïve vs SpatialGrid

Les mesures locales ont été réalisées avec un rayon de perception fixé à 50 et une taille de cellule de 40. Elles montrent que la grille spatiale devient rapidement plus rentable que la recherche exhaustive.

La méthode naïve valide correctement le modèle, mais elle devient rapidement coûteuse dès que N augmente. `SpatialGrid` réduit fortement le nombre de comparaisons, et l'écart devient très visible à $N = 800$, où la version optimisée est presque cinq fois plus rapide.

TABLE I
COMPARAISON ENTRE RECHERCHE NAÏVE ET GRILLE SPATIALE

N	Naïf (ms/step)	SpatialGrid (ms/step)	Speedup
100	0.194	0.130	1.50x
200	0.702	0.335	2.10x
400	2.843	0.835	3.40x
800	10.326	2.209	4.68x

B. MPI et montée en charge

Les mesures de strong scaling ont été exécutées avec $N = 5000$ boids et 30 étapes. Elles mettent en évidence un gain net lorsque le calcul est réparti sur plusieurs processus.

TABLE II
STRONG SCALING DE LA VERSION MPI ($N = 5000$, 30 ÉTAPES)

Processus	Séquentiel (ms)	MPI (ms)	Speedup
1	1291.104	1441.998	0.90x
2	1291.104	688.455	1.91x
4	1291.104	348.516	3.89x

Le résultat le plus important est obtenu à 4 processus : le temps MPI tombe à 348.516 ms, soit 3.89x plus rapide que la référence séquentielle mesurée dans le même scénario. Le cas à 1 processus reste légèrement plus lent que le séquentiel pur, ce qui correspond au coût d'infrastructure MPI.

Les résultats agrégés du fichier `scalability_results.csv` confirment cette tendance sur plusieurs tailles de problème : le passage de 500 à 5000 boids fait monter le speedup de 2.32x à 8.99x, ce qui montre que la version distribuée devient de plus en plus intéressante lorsque la charge augmente.

V. DÉFIS RENCONTRÉS ET RÉOLUTIONS

Nous avons rencontré plusieurs problèmes concrets pendant la mise au point et les campagnes de mesures.

A. Lancement MPI sur le cluster

Le premier blocage venait de l'utilisation de `srun`, qui provoquait des erreurs de type `timeout waiting for task launch`. La solution retenue a été d'utiliser `mpirun` directement dans les scripts SLURM, ce qui a stabilisé le lancement des exécutions distribuées.

B. Compatibilité binaire / environnement

Un incident de compatibilité `GLIBCXX` entre le binaire compilé localement et l'environnement du cluster a également été rencontré. Une recompilation complète avec les modules GCC et OpenMPI du cluster a résolu le problème et a permis d'obtenir des mesures reproductibles.

C. Coût artificiellement doublé

Le benchmark MPI lançait au départ une phase séquentielle puis une phase distribuée même lorsque plusieurs processus étaient utilisés. Cette logique doublait artificiellement le coût des runs parallèles. Elle a été corrigée en ne lançant la partie séquentielle que pour le cas $P = 1$.

D. Contraintes d'allocation

Certains tests à plusieurs processus ne disposaient pas d'assez de slots MPI lorsque SLURM ne réservait qu'une petite allocation. Pour les essais à deux processus, `-oversubscribe` a permis de valider le pipeline expérimental, mais les tests à quatre processus sont restés plus sensibles aux contraintes d'exécution et aux délais d'initialisation.

E. Benchmarks trop courts

Les mesures de weak scaling sur une seule étape restaient dominées par le coût d'initialisation du runtime Kokkos / MPI. Ce point explique la chute d'efficacité observée sur les tests très courts et justifie l'utilisation future de runs plus longs (10 à 100 étapes) pour lisser ce surcoût.

VI. BENCHMARKING COMPLET

Les mesures ont été collectées à partir de trois fichiers principaux : `bench.csv` pour la comparaison naïf / grille spatiale, `summary.csv` pour le strong scaling MPI, et `scalability_results.csv` pour la montée en charge sur plusieurs tailles de problème.

Les tendances observées sont cohérentes sur l'ensemble des campagnes. La grille spatiale réduit nettement le coût de voisinage, tandis que la version distribuée devient de plus en plus rentable lorsque le nombre de boids augmente. Sur le sweep global, le temps distribué passe de 39.310 ms pour 500 boids à 375.477 ms pour 5000 boids, avec un temps par step qui reste bien inférieur à la version séquentielle correspondante.

A. Visualisation des runs exploitables

Les figures suivantes ont été générées à partir des fichiers de sortie exploitables collectés sur le cluster (tests smoke et run multi-nud final).

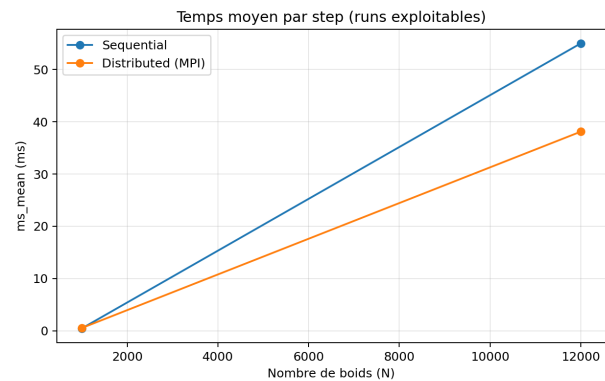


FIGURE 1. Temps moyen par step (séquentiel vs distribué) sur les runs exploitables

La Figure 1 montre le comportement attendu : sur petite taille ($N = 1000$), le coût d'infrastructure MPI domine encore ; sur une taille plus grande ($N = 12000$), la version distribuée devient plus rapide que la version séquentielle.

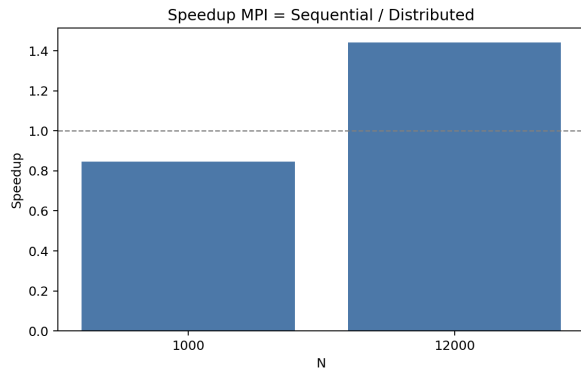


FIGURE 2. Speedup MPI (*Sequential / Distributed*)

La Figure 2 confirme ce point : le speedup passe d'environ $0.85\times$ à $N = 1000$ (pas de gain) à environ $1.44\times$ à $N = 12000$ (gain net). Cette transition valide l'intérêt de la distribution lorsque la charge de calcul est suffisante pour amortir les surcoûts de communication et de lancement.

VII. PLANIFICATION DES CAMPAGNES SUR CLUSTER

Conformément aux contraintes de la plateforme Zen, chaque campagne est planifiée en heures-curs avant soumission à l'encadrant. Le budget total du groupe est de 10000 heures-curs, et un nud complet (2 sockets $\times$ 64 curs) consomme 128 heures-curs par heure de walltime.

A. Méthode d'estimation du coût

Le coût prévisionnel d'un run est calculé par :

$$H_{\text{coeurs}} = T_h \times N_{\text{nodes}} \times N_{\text{tasks/node}} \times N_{\text{cpus/task}} \quad (6)$$

où T_h est la durée en heures.

B. Planning prévisionnel valable

La stratégie retenue est progressive : calibration mono-nud (1 cur), puis runs de scaling mono-nud (1/2/4 processus), puis un run multi-nud final.

TABLE III
PLANNING PRÉVISIONNEL ET COÛT ESTIMÉ

Run	Configuration	Durée	Coût total
RUN 1	1 nud, 1 task, 1 CPU	3 $\times$ 5 min	0.25 h-curs
RUN 2	1 nud, 1 task, 1 CPU	3 $\times$ 20 min	1.00 h-curs
RUN 3	1 nud, 4 tasks, 1 CPU	3 $\times$ 25 min	5.00 h-curs
RUN 4	1 nud, 4 tasks, 1 CPU	4 $\times$ 10 min	2.67 h-curs
RUN 5	2 nuds, 2 tasks/nud, 1 CPU	1 $\times$ 30 min	2.00 h-curs
Total	—	—	10.92 h-curs

C. Suivi de consommation

Le suivi est effectué avec `sreport user TopUsage` (fenêtre temporelle explicite), afin de comparer la consommation réelle au plan prévisionnel et d'ajuster les campagnes avant le run multi-nud final.

VIII. DISCUSSION

Les résultats montrent que les gains de performance ne dépendent pas seulement du nombre de processus, mais aussi de la taille du problème, du nombre d'étapes simulées et du coût d'initialisation. En pratique, la structure spatiale apporte déjà un premier gain algorithmique important, puis MPI permet de franchir un second palier lorsque la charge devient suffisante pour amortir les communications.

La principale limite des mesures actuelles reste leur fragilité sur les très petits runs. Les benchmarks courts capturent surtout le coût de lancement et d'initialisation, ce qui peut masquer le gain réel de la parallélisation. Pour une analyse finale plus robuste, il faudra répéter chaque expérience plusieurs fois et augmenter la durée des runs les plus courts.

IX. CONCLUSION

Les résultats obtenus confirment la pertinence des approches retenues. La version MPI atteint un speedup de $3,89\times$ à quatre processus pour $N = 5000$ boids, avec un surcoût mono-processus limité par rapport au séquentiel pur coût imputable à l'infrastructure MPI et non à une dégradation du modèle. Sur le sweep global, le speedup progresse de $2,32\times$ pour 500 boids jusqu'à $8,99\times$ pour 5000 boids, confirmant que l'approche distribuée devient de plus en plus avantageuse à mesure que la charge augmente. La grille spatiale, déjà introduite au premier semestre, reste le premier levier d'optimisation : elle réduit la complexité de voisinage de $O(N^2)$ à un comportement proche de $O(N)$ en moyenne, et produit à elle seule un gain de $4,68\times$ à $N = 800$.

Bilan global

Pris ensemble, les deux semestres forment un projet cohérent et progressif. Le premier semestre a posé les fondations : implémentation fidèle des trois règles de Reynolds, architecture modulaire séparant Boid, Flock et SpatialGrid, suite de tests unitaires GoogleTest, et interface SFML permettant la validation visuelle des comportements émergents. Cette base propre a directement rendu possible la parallélisation du second semestre, aussi bien sur le plan architectural la décomposition de domaine s'appuie naturellement sur la grille spatiale existante que sur le plan méthodologique, les benchmarks locaux servant de référence aux mesures distribuées.

Au terme de ces deux semestres, le projet livre trois implémentations opérationnelles et comparées : séquentielle, MPI avec échange de halos et migration inter-rangs, et Kokkos pour le parallélisme à mémoire partagée. La cohérence physique du modèle est préservée dans chacune d'elles : le wrapping toroïdal et la migration des boids aux frontières garantissent la continuité du comportement collectif quelle que soit la répartition des agents entre processus.

Réflexion sur le travail et son apport pédagogique

Ce projet illustre bien pourquoi la simulation de systèmes naturels constitue un terrain particulièrement riche pour aborder le calcul haute performance. La dynamique d'un banc de poissons est, de prime abord, un sujet accessible et intuitif ;

c'est précisément ce qui en fait un bon vecteur pédagogique. Derrière la simplicité apparente des trois règles de Reynolds se cachent des problèmes algorithmiques et architecturaux concrets : gestion des dépendances de données entre agents, compromis entre granularité de la décomposition et fréquence des communications, sensibilité des mesures de performance aux conditions d'exécution.

Ce second semestre a été dense à gérer, entre les cours, les autres projets et les contraintes techniques liées à l'environnement distribué. Nous n'avons pas pleinement exploité les ressources du cluster autant que nous l'aurions souhaité, et nous en assumons la responsabilité. Ces difficultés font partie intégrante de la réalité du HPC en conditions réelles, et les avoir traversées reste formateur, même si elles ont parfois limité l'ampleur des campagnes de mesures que nous aurions souhaité conduire.

En définitive, ce sujet a bien rempli son rôle pédagogique. Il a permis de relier des concepts théoriques complexité algorithmique, loi d'Amdahl, scalabilité faible et forte à une réalité pratique souvent plus complexe que les modèles ne le laissent supposer. L'intérêt du sujet n'a jamais fait défaut ; c'est peut-être ce qui a rendu d'autant plus frustrant de ne pas avoir pu le mener aussi loin qu'envisagé.

X. RÉFÉRENCES BIBLIOGRAPHIQUES

Cette section liste les sources scientifiques et techniques ayant servi de base au projet.

RÉFÉRENCES

- [1] C. W. Reynolds, "Flocks, herds and schools : A distributed behavioral model," in *Proc. SIGGRAPH*, 1987.
- [2] C. W. Reynolds, "Steering behaviors for autonomous characters," in *Proc. Game Developers Conf.*, 1999.
- [3] D. S. Hollman *et al.*, "Kokkos : Enabling manycore performance portability through polymorphic memory access patterns," *J. Parallel Distrib. Comput.*, vol. 74, no. 12, pp. 3202–3216, 2014.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI : Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 1998.
- [5] OpenMP Architecture Review Board, "OpenMP Application Program Interface," Version 5.2, 2023. [Online]. Available : <https://www.openmp.org/>
- [6] L. Gomila, "SFML - Simple and Fast Multimedia Library," 2024. [Online]. Available : <https://www.sfml-dev.org>
- [7] Google, "GoogleTest User's Guide," 2024. [Online]. Available : <https://google.github.io/googletest/>
- [8] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Comput. Conf.*, 1967, pp. 483–485.
- [9] M. Buckland, *Programming Game AI by Example*. Course Technology, 2004.
- [10] D. Shiffman, *The Nature of Code : Simulating Natural Systems with Processing*. 2012. [Online]. Available : <https://natureofcode.com/>
- [11] Kitware, "CMake Build System," 2024. [Online]. Available : <https://cmake.org/>